



VIT[®]
B H O P A L
www.vitbhopal.ac.in

VIT BHOPAL UNIVERSITY

School of Computing Science and Engineering

**Bhopal-Indore Highway, Kothrikalan, Sehore
Madhya Pradesh - 466114**

VITYARTHI COURSE PROJECT REPORT

REG.NO : 26BAI10538

NAME : Ansh Walia

BRANCH : CSE(AI/ML)

SEMESTER: Fall Semester 2026-27

Contents

S.NO	TITLE
1.	Introduction To The Project
2.	Problem Statement
3.	Objectives
4.	Function Requirement
5.	Non-Functional Requirements
6.	System Architecture
7.	Process Flow / Workflow Diagram
8.	Class / Component Diagram
9.	Sequence Diagram
10.	Data Strucutre Used
11.	Design Decisions and Rationale
12.	Source Code
13.	SS Of Output
14.	Challenges Faced
15.	Learning And Key Takeaways
16.	Future Enchancements

Introduction To The Project

Passwords are the most common way to secure personal or business account, but the majority of them are using weak ones and even telling users that their password is weak but not explaining why they should change it. The most common passwords are password1, qwerty123, etc.

The project, named Password strength checker is a command line Python app that checks the level of a given password and provides some advice on what should be changed in the given password in order to increase its strength. It also gives a score and a level to the password and checks it against the most common weak password lists, keyboard patterns, names, and some other criteria.

Problem

Statement

Most casual users reuse simple, memorable passwords because they receive no meaningful feedback beyond a generic "weak/strong" label. Basic strength meters found on many websites check only surface-level rules (length, one uppercase letter, one digit) and fail to catch passwords that are technically "complex" by those rules but still trivially guessable — for example "Qwerty123!" satisfies most naive rule sets while being a well-known weak pattern.

The problem this project addresses is the absence of a transparent, rule-explained password evaluator that a student or a small application could run locally, offline, and without any paid API or breach-database service, while still catching the most common real-world weaknesses.

Objectives

Evaluate a user-supplied password across length and four character-class categories (uppercase, lowercase, digit, special character).

Detect membership in a list of well-known weak/common passwords and known keyboard-walk sequences (e.g. "qwerty", "asdfgh").

Detect the presence of common first names inside the password, since names are a frequent, easily guessed component.

Detect three-or-more repeated consecutive characters, which weaken effective entropy even in long passwords.

Produce a bounded numeric security score (0–10), a human-readable strength label, and a visual progress bar.

Generate a list of specific, actionable improvement suggestions rather than a single generic verdict.

Persist the full evaluation as an audit entry to a local text file on request.

Function

Requirement

Module 1 – Input & Validation

Collects the user's name, registration number, and the password to be evaluated through input(). Calculates simple derived value (password length) used by all subsequent module.

Input: Name, registration number, password (all as text through input())

Output: Validated raw password string and its length, passed to Module 2

Processing: Len() calculation; counters and flags initialized

Module 2 – Strength Analysis Engine

The core of the application. Iterates through password once, classifying every character (uppercase, lowercase, digit, special character, whitespace) and runs five independent checks: common-password lookup, keyboard-pattern lookup, common-name lookup, repeated-character detection and first/last-character rules.

Input: Password string and lowercase form

Output: Character counts, boolean flags (has_upper, has_digit, is_common, has_keyboard_walk, has_name, has_repeats, etc.)

Processing: A single-pass for loop over characters; three separate for loops over lookup lists (common_passwords, keyboard_patterns, common_names); a loop comparing adjacent characters for repeats

Module 3 – Scoring, Reporting & Persistence

Converts the flags and counts from Module 2 into a bounded score (0-10) using additive rules for good practices and subtractive penalties for identified weaknesses, assigns a strength label, renders a text-based visual bar, prints a formatted report, and optionally appends the full result to saved_passwords.txt

Input: Counts and flags from Module 2

Output: Numeric score /10, strength label, visual bar, printed report, optional saved text-file entry

Processing: Scoring algorithm (additive rules for good practices, subtractive penalties for identified weaknesses), score clamped to [0, max_score], if/elif chain for the label, string-repetition for the bar, formatted print statements, conditional file.write() calls

Overall Workflow

The user launches the program, provides their name, registration number, and a password. The program then quietly analyzes the given password and prints out a complete assessment report in the console. The user is then asked whether they want to save the report, and based on their response, the report is either appended to a local text file or the program terminates without saving any data to the disk.

Non-Functional Requirements

Performance: The password is checked character by character (on $O(n)$ in password length) for character classes, and lookup tables are of fixed size (15 or fewer items), so the calculation speed is virtually independent of the input string length.

Usability: The results are broken down into several sections, with section headings separated by empty lines and values presented in fixed-length columns for better readability in a text console.

Reliability: The variable score is capped at `max_score` (10) regardless of the number of identified weaknesses; thus, even adding more checks would not break the score display logic.

Maintainability: Each of the common password checks (passwords dictionary, keyboard patterns, names, and repetitions) is implemented as a separate block of code that modifies the same set of flags. This approach allows each check to be modified or added independently without affecting others.

Error handling / Data integrity: The saved file is only written to if the user inputs y/n (case-insensitive), and a message is shown to the user if the save was canceled instead of failing silently.

Resource efficiency: The program only utilizes Python standard library resources (import os), does not require any external dependencies, network access, or significant memory or CPU time.

System Architecture

USER (Console Input: name, reg. no., password)



MODULE 1 – Input & Validation



MODULE 2 – Strength Analysis Engine

(character scan · common-password check ·
keyboard-pattern check ·

name check · repeat-character check · edge-
character rules)



MODULE 3 – Scoring, Reporting & Persistence

(score + label + visual bar → console report →
optional file save)



**OUTPUT: Console Report + saved_passwords.txt
(optional)**

Process Flow / Workflow Diagram

Initialize counters and boolean flags
(upper_count, lower_count, digit_count,
special_count, space_count and their
has_* flags)

1. Iterate through every character of
the password; classify and count it

2. Check the lowercased password
against the common_passwords list

3. Check the lowercased password
against the keyboard_patterns list

4. Check the lowercased password
against the common_names list

5. Scan adjacent characters for three-
or-more repeats

6. Check whether the password starts
with a letter and ends with a special
character

- 7. Calculate the score from length, character variety, balance bonus, and penalties**
- 8. Map the final score to a strength label (Very Weak → Very Strong)**
- 9. Build the text-based visual score bar**
- 10. Print the full evaluation report and any actionable suggestions**
- 11. Ask the user whether to save the report; append to saved_passwords.txt if "yes"**

Class / Component Diagram

The program is implemented as a single procedural script instead of Python classes, which is suitable for a basic-Python course project. Instead of a class diagram, the components below represent the logical groupings that the code is organized into.

InputComponent — collects name, reg. no., password — computes length ↓

AnalysisComponent — character classifier

(upper/lower/digit/special/space) — **CommonPasswordCheck** ·

KeyboardPatternCheck · **NameCheck** · **RepeatCheck** · **EdgeCharCheck** ↓

ScoringComponent — additive length/variety scoring — balance bonus — penalty deductions — score clamp (0–10) + strength label ↓

ReportComponent — visual bar builder — console report printer —

FileHandler: append audit entry to **saved_passwords.txt**

Sequence Diagram

User → Program: Runs psc.py

Program → User: Prompts for name, registration number, password

User → Program: Supplies name, registration number, password

Program → AnalysisComponent: Password passed in for character classification and pattern checks

AnalysisComponent → ScoringComponent: Counts and boolean flags passed in for scoring

ScoringComponent → ReportComponent: Final clamped score, strength label, and feedback list passed in

ReportComponent → User: Full evaluation report printed to the console

Program → User: Prompts "Do you want to save this report to a file? (yes/no)"

User → Program: Answers yes/no

Program → saved_passwords.txt: (If yes) Appends the audit entry to the file and confirms to the User

Data Structure

Used

This project stores no persistent structured data beyond a plain append-only text log, so a relational ER diagram does not apply. The structures below serve the same organizing role a schema would.

common_passwords (list of str) — 15 well-known weak passwords checked against the input

keyboard_patterns (list of str) — 5 common keyboard-walk sequences

common_names (list of str) — 7 common first names checked as password substrings

feedback (list of str) — accumulates every actionable suggestion generated during scoring

upper_count / lower_count / digit_count / special_count / space_count (int) — per-character-class counts

has_upper / has_lower / has_digit / has_special / has_spaces (bool) — quick-check flags

is_common / has_keyboard_walk / has_name / has_repeats (bool) — results of the four weakness checks

score / max_score (int) — running and maximum security score, clamped to [0, 10]

visual_bar (str) — text-based progress bar built from "#" and "-" characters

Design Decisions and Rationale

13.1 Rule-based scoring instead of a machine-learning model

A rule-based approach was chosen deliberately over a trained classifier. CSA1021 is a foundational, basic-Python course covering control flow, strings, lists, and files — not machine-learning libraries — so a solution built entirely from if/elif logic and list lookups demonstrates the actual syllabus concepts being assessed.

13.2 List-based lookups over regular expressions

Common passwords, keyboard patterns, and names are checked with simple "in" membership tests against Python lists rather than regular expressions, keeping the logic readable and directly traceable to core-course concepts.

13.3 Additive and subtractive scoring instead of pass/fail

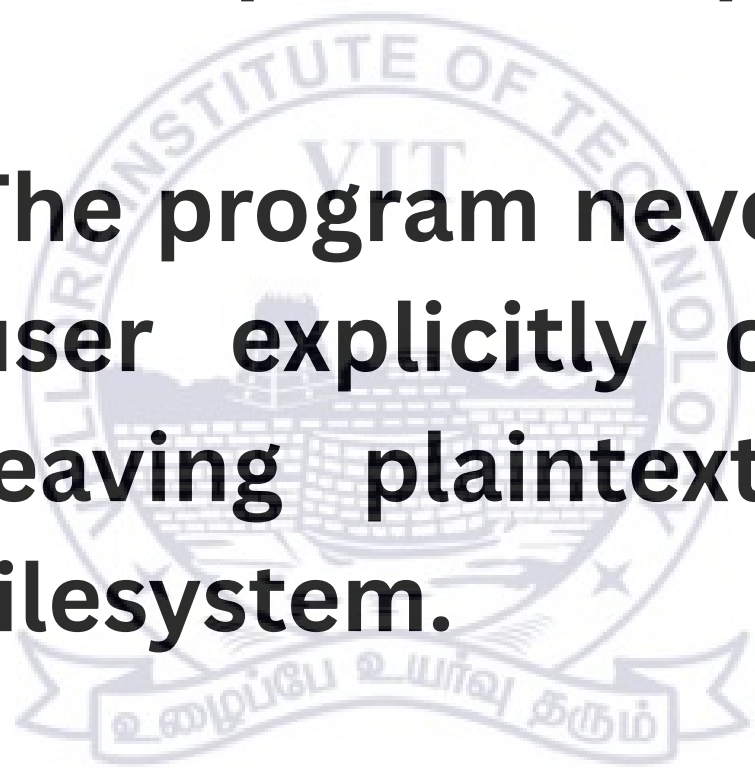
The program awards points for each good property and subtracts points for each detected weakness, producing a graded 0–10 score that better reflects real password strength than a binary verdict.

13.4 Independent, separable detection steps

Each weakness check is implemented as its own self-contained block operating on shared flags rather than one large nested if-chain – a new rule can be added as a new block without touching existing checks.

13.5 Optional, explicit file persistence

The program never writes to disk unless the user explicitly opts in, avoiding silently leaving plaintext password data on the filesystem.



Source Code

```
import os
print("=" * 55)
print("    PASSWORD STRENGTH CHECKER")
print("=" * 55)
special_characters = "!@#$%^&*(),.?\"':{}|<>-=_+[]\\;/`~"
common_passwords = [
    "password",
    "123456",
    "12345678",
    "qwerty",
    "admin",
    "welcome",
    "12345",
    "football",
    "monkey",
    "letmein",
    "dragon",
    "master",
    "sunshine",
    "princess",
    "solo"
]
keyboard_patterns = [
    "qwerty",
    "asdfgh",
    "zxcvbn",
    "123456",
    "654321"
]
```

```
common_names =
["john","alex","david","emma","smith","mike","sam"]
password = input("Enter a password to evaluate: ")
name=input("Enter YOur Name: ")
regno=(input("Enter Your Registration Number: "))
length = len(password)
upper_count = 0
lower_count = 0
digit_count = 0
special_count = 0
space_count = 0
has_upper = False
has_lower = False
has_digit = False
has_special = False
has_spaces = False
for char in password:
    if char.isupper():
        upper_count = upper_count + 1
        has_upper = True
    elif char.islower():
        lower_count = lower_count + 1
        has_lower = True
    elif char.isdigit():
        digit_count = digit_count + 1
        has_digit = True
    elif char in special_characters:
        special_count = special_count + 1
        has_special = True
    elif char.isspace():
        space_count = space_count + 1
        has_spaces = True
password_lower = password.lower()
is_common = False
```

```
found_common_word = ""
for common in common_passwords:
    if common in password_lower:
        is_common = True
        found_common_word = common
        break
has_keyboard_walk = False
found_pattern = ""
for pattern in keyboard_patterns:
    if pattern in password_lower:
        has_keyboard_walk = True
        found_pattern = pattern
        break
has_name = False
found_name = ""
for name in common_names:
    if name in password_lower:
        has_name = True
        found_name = name
        break
has_repeats = False
consecutive_count = 0
for i in range(len(password) - 1):
    if password[i] == password[i + 1]:
        consecutive_count = consecutive_count + 1
        if consecutive_count >= 2:
            has_repeats = True
            break
    else:
        consecutive_count = 0
starts_with_letter = False
ends_with_special = False
if length > 0:
    if password[0].isalpha():
```

```
    starts_with_letter = True
    if password[-1] in special_characters:
        ends_with_special = True
score = 0
feedback = []
if length >= 8:
    score = score + 1
else:
    feedback.append("Password is too short. Must be at
least 8 characters long.")
if length >= 12:
    score = score + 1
if length >= 16:
    score = score + 1
if has_upper:
    score = score + 1
else:
    feedback.append("Include at least one uppercase
letter (A-Z).")
if has_lower:
    score = score + 1
else:
    feedback.append("Include at least one lowercase
letter (a-z).")
if has_digit:
    score = score + 1
else:
    feedback.append("Include at least one numeric digit
(0-9).")
if has_special:
    score = score + 1
else:
```

```
feedback.append("Include at least one special
character (!@#$%^&*).")
if upper_count >= 2 and lower_count >= 2 and
digit_count >= 2:
    score = score + 1
if is_common:
    score = score - 2
feedback.append("Warning: Contains a very common
password pattern (" + found_common_word + ").")
if has_keyboard_walk:
    score = score - 1
feedback.append("Warning: Contains a sequential
keyboard pattern (" + found_pattern + ").")
if has_name:
    score = score - 1
feedback.append("Warning: Contains an easily
guessable name (" + found_name + ").")
if has_repeats:
    score = score - 1
feedback.append("Warning: Contains 3 or more
identical characters in a row.")
if has_spaces:
    feedback.append("Notice: Password contains
whitespace characters.")
max_score = 10
if score < 0:
    score = 0
if score > max_score:
    score = max_score
if score <= 2:
    strength = "Very Weak"
elif score <= 4:
    strength = "Weak"
```

```
elif score <= 6:
    strength = "Moderate"
elif score <= 8:
    strength = "Strong"
else:
    strength = "Very Strong"
bar_total_blocks = 10
filled_blocks = score
empty_blocks = bar_total_blocks - filled_blocks
visual_bar = "[" + ("#" * filled_blocks) + ("-" *
empty_blocks) + "]"
print("\n" + "=" * 55)
print("          EVALUATION REPORT")
print("=" * 55)
print("Password Length      :", length)
print("Uppercase Count      :", upper_count)
print("Lowercase Count      :", lower_count)
print("Digit Count           :", digit_count)
print("Special Char Count    :", special_count)
print("Whitespace Count     :", space_count)
print("-" * 55)
print("Security Score        :", score, "/", max_score)
print("Score Visualizer      :", visual_bar)
print("Strength Rating       :", strength)
print("-" * 55)
if len(feedback) > 0:
    print("Actionable Suggestions:")
    for tip in feedback:
        print("  *", tip)
else:
    print("Excellent! Your password passed all
security checks.")
print("=" * 55)
save_option = input("\nDo you want to save this report to
a file? (yes/no): ")
if save_option.lower() == "yes" or save_option.lower() ==
"y":
    file_name = "saved_passwords.txt"
```

```
file.write("=" * 50 + "\n")
file.write("Name: " + name + "\n")
file.write("Registration Number: " + regno)
file.write("Evaluated Password : " + password + "\n")
file.write("Length          : " + str(length) + "\n")
file.write("Uppercase Letters : " + str(upper_count) +
"\n")
file.write("Lowercase Letters : " + str(lower_count) +
"\n")
file.write("Numeric Digits   : " + str(digit_count) +
"\n")
file.write("Special Characters : " + str(special_count) +
"\n")
file.write("Score           : " + str(score) + "/" +
str(max_score) + "\n")
file.write("Strength Rating  : " + strength + "\n")
file.write("Visual Meter     : " + visual_bar + "\n")
if len(feedback) > 0:
    file.write("\nIssues & Feedback:\n")
    for tip in feedback:
        file.write(" - " + tip + "\n")
else:
    file.write("\nStatus: Meets all requirements.\n")

file.write("=" * 50 + "\n\n")
file.close()
print("[+] Report successfully saved to", file_name)
else:
    print("[-] Result was not saved.")
```

Output

ScreenShots

```
=====
                        PASSWORD STRENGTH CHECKER
=====
Enter Your Name: Ansh Walia
Enter Your Registration Number: 26BAI10538
Enter a password to evaluate: HeyIt$J10

=====
                        EVALUATION REPORT
=====
Password Length      : 9
Uppercase Count      : 3
Lowercase Count      : 3
Digit Count          : 2
Special Char Count   : 1
Whitespace Count     : 0

-----
Security Score       : 6 / 10
Score Visualizer     : [#####----]
Strength Rating      : Moderate

-----
Excellent! Your password passed all security checks.
=====

Do you want to save this report to a file? (yes/no): yes
[+] Report successfully saved to saved_passwords.txt
PS C:\Users\Ansh walia\OneDrive\Documents\Codes> █
```

```
=====
                        PASSWORD STRENGTH CHECKER
=====
Enter Your Name: Rocky
Enter Your Registration Number: 7264
Enter a password to evaluate: 123123

=====
                        EVALUATION REPORT
=====
Password Length      : 6
Uppercase Count      : 0
Lowercase Count      : 0
Digit Count          : 6
Special Char Count   : 0
Whitespace Count     : 0

-----
Security Score       : 1 / 10
Score Visualizer     : [#-----]
Strength Rating      : Very Weak

-----
Actionable Suggestions:
* Password is too short. Must be at least 8 characters long.
* Include at least one uppercase letter (A-Z).
* Include at least one lowercase letter (a-z).
* Include at least one special character (!@#$$%^&*).

=====

Do you want to save this report to a file? (yes/no): yes
[+] Report successfully saved to saved_passwords.txt
PS C:\Users\Ansh walia\OneDrive\Documents\Codes> █
```

```
=====
                        PASSWORD STRENGTH CHECKER
=====

Enter YOur Name: Ansh
Enter Your Regisration Number: 52
Enter a password to evaluate: Ansh52I$Grea1

=====
                        EVALUATION REPORT
=====

Password Length      : 13
Uppercase Count      : 3
Lowercase Count      : 6
Digit Count          : 3
Special Char Count   : 1
Whitespace Count     : 0

-----

Security Score       : 7 / 10
Score Visualizer     : [#####---]
Strength Rating      : Strong

-----

Excellent! Your password passed all security checks.

=====

Do you want to save this report to a file? (yes/no): yes
[+] Report successfully saved to saved_passwords.txt
PS C:\Users\Ansh walia\OneDrive\Documents\Codes> █
```

```
=====
                        PASSWORD STRENGTH CHECKER
=====

Enter YOur Name: Ramu
Enter Your Regisration Number:
Enter a password to evaluate: oggyi$_Goa1

=====
                        EVALUATION REPORT
=====

Password Length      : 11
Uppercase Count      : 1
Lowercase Count      : 7
Digit Count          : 1
Special Char Count   : 2
Whitespace Count     : 0

-----

Security Score       : 5 / 10
Score Visualizer     : [#####-----]
Strength Rating      : Moderate

-----

Excellent! Your password passed all security checks.

=====

Do you want to save this report to a file? (yes/no): yes
[+] Report successfully saved to saved_passwords.txt
PS C:\Users\Ansh walia\OneDrive\Documents\Codes> █
```

```
=====
                        PASSWORD STRENGTH CHECKER
=====

Enter YOur Name: Billu Don
Enter Your Regisration Number: 25
Enter a password to evaluate: Billu_Ke_Changu1_se_n@_bach_paoge

=====
                        EVALUATION REPORT
=====

Password Length      : 33
Uppercase Count      : 3
Lowercase Count      : 22
Digit Count          : 1
Special Char Count   : 7
Whitespace Count     : 0

-----

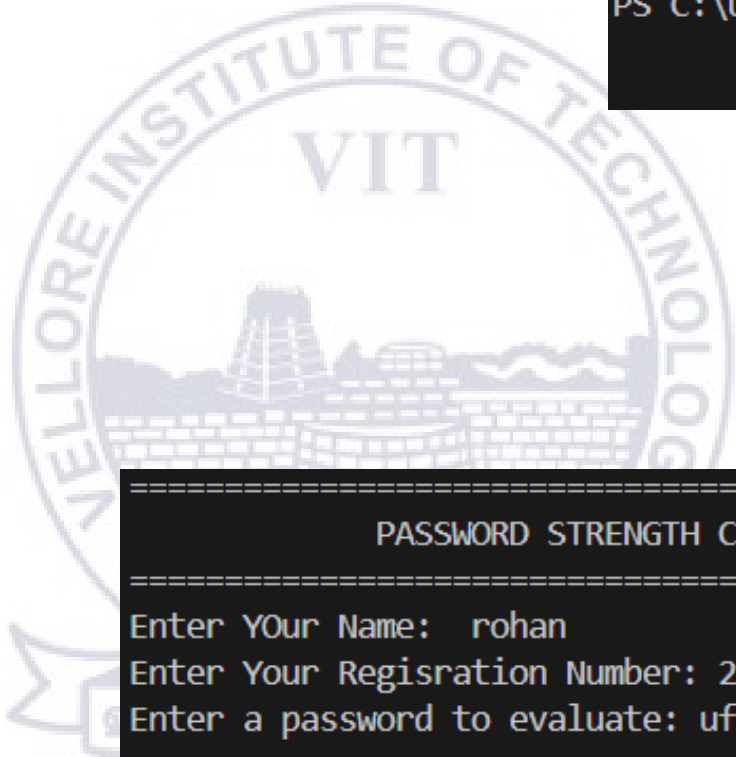
Security Score       : 7 / 10
Score Visualizer     : [#####---]
Strength Rating      : Strong

-----

Excellent! Your password passed all security checks.

=====

Do you want to save this report to a file? (yes/no): yes
[+] Report successfully saved to saved_passwords.txt
PS C:\Users\Ansh walia\OneDrive\Documents\Codes> █
```



VIT
PAL
nopal.ac.in

```
=====
                        PASSWORD STRENGTH CHECKER
=====

Enter YOur Name: rohan
Enter Your Regisration Number: 26bai10384
Enter a password to evaluate: ufxe9248 @3999

=====
                        EVALUATION REPORT
=====

Password Length      : 14
Uppercase Count      : 0
Lowercase Count      : 4
Digit Count          : 8
Special Char Count   : 1
Whitespace Count     : 1

-----

Security Score       : 4 / 10
Score Visualizer     : [####-----]
Strength Rating      : Weak

-----

Actionable Suggestions:
* Include at least one uppercase letter (A-Z).
* Warning: Contains 3 or more identical characters in a row.
* Notice: Password contains whitespace characters.

=====

Do you want to save this report to a file? (yes/no): yes
[+] Report successfully saved to saved_passwords.txt
PS C:\Users\Ansh walia\OneDrive\Documents\Codes> █
```

```
PS C:\Users\Ansh walia\OneDrive\Documents\Codes> & "C:\Program Files\Python313\pyt
=====
                        PASSWORD STRENGTH CHECKER
=====
Enter a password to evaluate: It$_0h1a1a
Enter Your Name: Bhavya
Enter Your Registration Number: 52

=====
                        EVALUATION REPORT
=====
Password Length      : 10
Uppercase Count      : 2
Lowercase Count      : 4
Digit Count          : 2
Special Char Count   : 2
Whitespace Count     : 0
-----
Security Score       : 6 / 10
Score Visualizer     : [#####----]
Strength Rating      : Moderate
-----
Excellent! Your password passed all security checks.
=====

Do you want to save this report to a file? (yes/no): no
[-] Result was not saved.
PS C:\Users\Ansh walia\OneDrive\Documents\Codes> |
```

```
PS C:\Users\Ansh walia\OneDrive\Documents\Codes> & "C:\Program Fi
=====
                        PASSWORD STRENGTH CHECKER
=====
Enter a password to evaluate: $tr0nG_P@s8w0rd
Enter Your Name: Anubhav
Enter Your Registration Number: 25

=====
                        EVALUATION REPORT
=====
Password Length      : 15
Uppercase Count      : 2
Lowercase Count      : 7
Digit Count          : 3
Special Char Count   : 3
Whitespace Count     : 0
-----
Security Score       : 7 / 10
Score Visualizer     : [#####---]
Strength Rating      : Strong
-----
Excellent! Your password passed all security checks.
=====

Do you want to save this report to a file? (yes/no): yes
```

```
PS C:\Users\Ansh walia\OneDrive\Documents\Codes> & "C:\Program Files\Python313\python.exe"
=====
                        PASSWORD STRENGTH CHECKER
=====
Enter a password to evaluate: A
Enter Your Name: a
Enter Your Registration Number: 26bai10538

=====
                        EVALUATION REPORT
=====
Password Length      : 1
Uppercase Count      : 1
Lowercase Count      : 0
Digit Count          : 0
Special Char Count   : 0
Whitespace Count     : 0
-----
Security Score       : 1 / 10
Score Visualizer     : [#-----]
Strength Rating      : Very Weak
-----
Actionable Suggestions:
* Password is too short. Must be at least 8 characters long.
* Include at least one lowercase letter (a-z).
* Include at least one numeric digit (0-9).
* Include at least one special character (!@#$$%^&*).
=====

Do you want to save this report to a file? (yes/no): yes
[+] Report successfully saved to saved_passwords.txt
PS C:\Users\Ansh walia\OneDrive\Documents\Codes> █
```



```
=====
                        PASSWORD STRENGTH CHECKER
=====
Enter a password to evaluate:
Enter Your Name:
Enter Your Registration Number:

=====
                        EVALUATION REPORT
=====
Password Length      : 0
Uppercase Count      : 0
Lowercase Count      : 0
Digit Count          : 0
Special Char Count   : 0
Whitespace Count     : 0
-----
Security Score       : 0 / 10
Score Visualizer     : [-----]
Strength Rating      : Very Weak
-----
Actionable Suggestions:
* Password is too short. Must be at least 8 characters long.
* Include at least one uppercase letter (A-Z).
* Include at least one lowercase letter (a-z).
* Include at least one numeric digit (0-9).
* Include at least one special character (!@#$$%^&*).
=====

Do you want to save this report to a file? (yes/no): no
[-] Result was not saved.
PS C:\Users\Ansh walia\OneDrive\Documents\Codes> █
```

Challenges Faced

Keeping the score within a reasonable 0–10 range ensured by clamping it before assigning the strength label
defining what goes and what does not go into each lookup list
with sufficient conciseness to allow quick scanning and actual use in practice
while avoiding false positives through the potentially dangerous use of substring matching
is accepted as an inevitable compromise in favor of simplicity and teaching adequacy.
Balancing the need for informative feedback with the desire for a compact console output dictated the decision to accumulate messages in a list and print this list as a titled block.

Learning And Key Takeaways

Turned a vague real-world requirement into a concrete testable set of rules, using only core Python constructs.

Further reinforced string traversal/classification methods (isupper, islower, isdigit, isspace, isalpha) with counters and flags.

Practiced script structure – separated,
understandable steps.

Learned the value of limiting a computed value (clamping) before downstream use.

Gained experience with file persistence (appending) and the need to gain user consent before writing.

Future Enhancements

A simple GUI created in Python's built-in tkinter.

Optional breach-database API integration (e.g. Have I Been Pwned's k-anonymity endpoint) without transmitting the full password.

A companion password-generator feature.

Loading the lookup lists from external text files rather than having them hardcoded.

Multi-language support for console messages.

*Thank
You*